

Execution plan

Excellent.

Now we convert everything into an **execution protocol** — structured as 10 sequential implementation phases.

Each phase:

- Has a clear goal
- Has a measurable output
- Has a validation checkpoint
- Ends with: "Type NEXT to continue."

This way your AI agent runs it like a controlled pipeline.

10-STEP EXECUTION PLAN

Audio-Based Sound Classification NN (From Scratch in C++)

◆ STEP 1 — Project Skeleton Setup

Goal

Create full project directory structure and empty class files.

Tasks

1. Create folder structure:
 - data/
 - include/
 - src/
 - models/
 - results/
2. Create header + cpp files:
 - AudioLoader
 - FeatureExtractor
 - Preprocessor
 - NeuralNetwork
 - Trainer
 - Evaluator

- Utils
- main.cpp

3. Set up CMake or Makefile.

✓ Validation

- Project compiles successfully.
- Empty program runs without crash.

🤖 Agent Instruction

When structure compiles:

Step 1 complete. Type NEXT to proceed to Step 2.

◆ STEP 2 — Implement WAV Audio Loader

🎯 Goal

Load 16kHz mono 2-second WAV and return vector of 32,000 samples.

Tasks

- Parse WAV header.
- Validate sampling rate.
- Normalize PCM values to $[-1, 1]$.
- Return vector.

✓ Validation

- Load sample file.
- Print sample count = 32000.
- Print first 5 values.

🤖 Agent Instruction

If output correct:

AudioLoader working. Type NEXT to proceed to Step 3.

◆ STEP 3 — Implement Feature Extraction

🎯 Goal

Convert raw samples → 6-feature vector.

Tasks

Compute:

- Total energy
- Mean amplitude
- Signal variance
- Zero Crossing Rate
- Mean frame energy
- Variance frame energy

Frame size = 1600 samples (20 frames).

 Validation

- Run on 1 clap file.
- Print 6 feature values.
- Ensure no NaN or extreme values.

 Agent Instruction

If features computed correctly:

Feature extraction complete. Type NEXT to proceed to Step 4.

◆ **STEP 4 — Dataset Builder & Label Encoding**

 Goal

Load all files from data folders and build dataset.

Tasks

- Traverse data/clap, knock, noise, silence
- Extract features for each
- Assign one-hot label
- Store in dataset structure

 Validation

- Print dataset size (~160–200).
- Print 1 sample feature + label.

 Agent Instruction

If dataset built:

Dataset created. Type NEXT to proceed to Step 5.

◆ STEP 5 — Implement Preprocessor (Normalization)

Goal

Fit normalization on training features only.

Tasks

- Compute mean & std per feature.
- Implement transform().
- Apply normalization to dataset.

Validation

- Print normalized feature.
- Mean approx 0.
- Std approx 1.

Agent Instruction

If normalization correct:

Normalization working. Type NEXT to proceed to Step 6.

◆ STEP 6 — Implement Neural Network (Forward Only)

Goal

Implement forward pass completely.

Architecture:

$6 \rightarrow 12 \rightarrow 8 \rightarrow 4$

ReLU + Softmax

Tasks

- Initialize weights (He initialization)
- Implement matrix multiplication
- Implement ReLU
- Implement Softmax

Validation

- Feed dummy input.
- Output size = 4.
- Sum of outputs ≈ 1 .

Agent Instruction

If forward pass works:

Forward pass verified. Type NEXT to proceed to Step 7.

◆ STEP 7 — Implement Backpropagation

Goal

Implement full backward pass + weight updates.

Tasks

- Compute output gradient ($A3 - Y$)
- Backprop through hidden layers
- Update weights with learning rate

Validation

- Loss decreases after multiple updates on same sample.
- No exploding gradients.

Agent Instruction

If training reduces loss:

Backpropagation working. Type NEXT to proceed to Step 8.

◆ STEP 8 — Implement Training Loop

Goal

Train model across full dataset.

Tasks

- Shuffle training set
- Loop for 200–300 epochs
- Track:
 - Training loss
 - Training accuracy
 - Validation accuracy

Validation

- Training accuracy increases over epochs.
- Validation accuracy reasonable (>60% expected initially).

Agent Instruction

If training stable:

Training engine complete. Type NEXT to proceed to Step 9.

◆ **STEP 9 — Implement Evaluation & Model Saving**

Goal

Test on test set and save trained model.

Tasks

- Compute test accuracy.
- Build confusion matrix.
- Save:
 - W1, W2, W3
 - b1, b2, b3
 - normalization parameters

Validation

- Test accuracy printed.
- Model file saved in models/.

Agent Instruction

If evaluation successful:

Evaluation complete. Type NEXT to proceed to Step 10.

◆ **STEP 10 — Inference Mode**

Goal

Allow user to classify new WAV file.

Tasks

- Load saved model.
- Load saved normalization parameters.
- Extract features.
- Normalize.
- Forward pass.

- Print predicted class + probabilities.

Validation

- Classifies known test files correctly.
- Outputs probability distribution.

Agent Final Output

If successful:

Project complete. Model ready for demonstration.

End State

At completion:

- End-to-end pipeline works
- Model trained
- Test accuracy measured
- Inference functional
- Modular architecture maintained